Smart-X IoT Gateway & Diagnostics Engine

The Smart-X IoT Gateway is a two-project .NET solution designed for real-time telemetry ingestion, diagnostic alerting, and configuration management across ESP32 and IoT sensor networks. The architecture splits responsibilities between a WPF desktop client and an ASP.NET Core Web API backend.

Solution Architecture

```
SmartXApp.sln

— SmartX.Api/      < Backend ASP.NET Core Web API
— Controllers/     < REST API endpoints
— Uploads/         < Storage directory for configuration/log attachments
— Program.cs       < API pipeline & Swagger configuration

— SmartXApp/       < Frontend WPF Desktop Application (MVVM)
— Models/          < TelemetryPacket<T>, MeterPayload, ZoneNode
— Services/        < BatchProcessorService, TelemetrySimulator
— ViewModels/      < MainViewModel data-binding context
— Views/           < MainWindow layout with 3-Pillar menu
```

Core Technical Features

- **Generic Telemetry Streams (TelemetryPacket<T>)**: Utilizes C# generics to avoid boxing/unboxing overhead when transmitting numeric, boolean, or complex sensor readings.
- **Operator Overloading (MeterPayload)**: Implements overloaded +, -, >, and < operators to allow direct load aggregation and delta evaluation between smart utility meters.

- **Recursive Tree Traversal (ZoneNode)**: Evaluates nested, multi-tier deployment zones via deep tree recursion (ValidateZoneRecursive).
- **Jagged Array Flattening (BatchProcessorService)**: Processes and flattens multi-dimensional raw historical batches (double[][]) into clean generic collections.
- **Three-Pillar UI Navigation**: A modular WPF dashboard shell featuring Pillar 1 (Telemetry Ingestion & Registration active), with Pillars 2 and 3 safely disabled for Part 2 scope compliance.

REST API Specifications

Method	Endpoint	Description
POST	/api/telemetry/numeric	Ingests numeric generic telemetry packets (TelemetryPacket<double>).
POST	/api/telemetry/boolean	Ingests binary state telemetry packets (TelemetryPacket<bool>).
POST	/api/sensors/upload-config	Accepts multi-part (multipart/form-data) sensor logs and JSON config files.

Setup & Running Instructions

Prerequisites

- .NET 8.0 SDK
- Visual Studio 2022 (with *.NET Desktop* and *ASP.NET/Web Development* workloads)

Quick Start

1. **Clone the repository:**

```bash
git clone https://github.com/PMUTAPATI/SmartXApp.git
cd SmartXApp
```

2. **Run the ASP.NET Core Web API:**

```bash
cd SmartX.Api
dotnet run
```

Swagger UI will be accessible at http://localhost:5000/swagger.

3. **Run the WPF Client Dashboard:**

- Open SmartXApp.sln in Visual Studio 2022.
- Right-click SmartXApp in Solution Explorer and select Set as Startup Project.
- Press F5 to start the desktop engine.

Docker Containerization

To run the Web API backend inside a Docker container:

```bash
docker build -t smartx-api -f SmartX.Api/Dockerfile .
docker run -d -p 5000:80 --name smartx-api-container smartx-api
```

Code References

- **C# Generics & Type Safety**: Microsoft Learn. *Generics in C# (C# Programming Guide)*. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/generics/
- **Operator Overloading**: Microsoft Learn. *Operator Overloading - Define Overloaded Operators in C#*. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/operator-overloading
- **ASP.NET Core Web API & File Uploads**: Microsoft Learn. *Upload Files in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/file-uploads
- **Jagged Arrays in C#**: Microsoft Learn. *Jagged Arrays (C# Programming Guide)*. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/arrays/jagged-arrays
- **WPF Data Binding & MVVM**: Troelsen, A. and Japikse, P., 2022. *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*. 11th ed. New York: Apress.